

@andrian.yablonskyy/thub-agent

The Agent CLI (**thub**) for **TestHub** — a self-hosted job network that lets CI/CD pipelines and individual developers run firmware tests on real hardware or emulators in a private lab. **thub** is the single entry point for both: it's stateless, everything it knows comes from the **Coordinator** API, and it runs identically on a GitHub-hosted runner and a developer laptop — a developer reproducing a CI failure runs exactly the same command the pipeline runs.

See the [main TestHub repo](#) for the full system architecture and how this fits with the Coordinator and the **Client**.

Install

```
npm i -g @andrian.yablonskyy/thub-agent
# or, one-off in CI:
npx -y @andrian.yablonskyy/thub-agent run --type sw --image
"$IMAGE_URL" --tests "$TESTS_URL" --wait
```

Configuration

Read from flags, then environment (**THUB_URL**, **THUB_TOKEN**, **THUB_GROUP**, **THUB_USER**), then `~/.config/thub/agent.json`, then a bundled default. **url/token** are required by the time a command actually talks to the Coordinator; **group/user** are optional everywhere.

`npm install -g` creates `~/.config/thub/agent.json` for you (blank **url/token**, so nothing works until you set them) if it doesn't already exist — a re-install never overwrites it. Fill it in with **thub config set**:

```
thub config set url https://thub.example.com
thub config set token agt_...
thub config set group 548ae4ae-... # optional default --group
thub config set user "Your Name" # optional default --user
```

Commands

<code>thub run</code>	<code>[options]</code>	Submit a test job and follow its log
<code>thub status</code>	<code><jobId></code>	Show status; follow log if running, show artifacts if done
<code>thub cancel</code>	<code><jobId></code>	Cancel a job
<code>thub resources</code>		List resources and their status
<code>thub jobs</code>	<code>[--mine] [--state <s>]</code>	List recent jobs
<code>thub config set</code>	<code><key> <value></code>	Save coordinator URL / token / default group / default user locally
<code>thub check-update</code>		Compare this Agent with the

```
latest published version
thub self-update [--to <x.y.z>]      Update this Agent with npm
i -g
thub --version
```

Self-update. An admin can request an update for this agent (or all agents) on the Coordinator's Agents page. The next command that talks to the Coordinator then installs it with `npm i -g` (retrying through `sudo` on an interactive terminal) and re-runs itself on the new version. If the install fails — e.g. no permission in CI — it prints the manual command and carries on with the current version; a run never fails because of an update. Set `THUB_NO_SELF_UPDATE=1` to opt out.

Key options for `thub run`:

Option	Description
<code>--type hw sw</code>	Required resource type.
<code>--board <name> / --label <l></code>	Required labels (repeatable).
<code>--group <groupId></code>	Restrict scheduling to resources that are members of this group. Falls back to <code>THUB_GROUP</code> / <code>thub config set group <id></code> .
<code>--user <name></code>	Free-text job owner — a label, not an identity. Falls back to <code>THUB_USER</code> / <code>thub config set user <name></code> .
<code>--image <url></code>	Firmware/build image URL, fetched by the Client.
<code>--sha256 <hex></code>	Expected sha256 of <code>--image</code> ; the Client verifies it before flashing/running.
<code>--tests <url> / --suite <name></code>	Test package and suite.
<code>--arg <value></code>	Extra argument passed through to <code>run-tests.sh</code> on the Client (repeatable).
<code>--timeout <dur></code>	e.g. <code>30m</code> , default <code>30m</code> .
<code>--priority <n></code>	0–100; CI defaults to 50, CLI to 60 so a developer is not starved by a busy pipeline.
<code>--meta <key=value></code>	Arbitrary metadata stored on the job (repeatable) — CI job ids, git coordinates, anything else worth attaching to the run.
<code>--source ci cli</code>	Override auto-detected job source (defaults to <code>ci</code> when <code>\$GITHUB_ACTIONS=true</code> , else <code>cli</code>).
<code>--dry-run</code>	Exercise the full pipeline without the Client executing anything for real.
<code>--wait</code>	Do not detach on job end; exit with the job's verdict code (used in CI).

Option	Description
<code>--detach</code>	Print the job id and exit immediately.
<code>--json</code>	Machine-readable output.
<code>thub run</code> prints the job ID, then streams logs until Ctrl-C. Ctrl-C detaches; the job keeps running on the Client. <code>thub status <jobId></code> prints the current state; if active it keeps streaming, if done it prints the verdict and artifact download links.	

Exit codes make the Agent usable as a CI step:

Code	Meaning
0	PASSED
1	FAILED
2	ERROR, TIMEOUT, or LOST
3	CANCELED
4	Usage, auth or connection error
130	Detached with Ctrl-C (job still running)

Examples

Associate a CI/CD job id with the internal job id:

```
thub run --type sw --image "$IMAGE_URL" --tests "$TESTS_URL" \
  --meta ciJobId="$GITHUB_RUN_ID" --wait
thub status A-00123 --json | jq '.spec.meta.ciJobId'
```

Pass Git repo/branch/hash/tag to the Client — these travel as `--meta key=value` and the Client exposes each one to `run-tests.sh` as `THUB_META_<KEY>` (`ciJobId` → `THUB_META_CI_JOB_ID`):

```
thub run --type hw --board nucleo-f401re \
  --image "$IMAGE_URL" --tests "$TESTS_URL" --suite smoke \
  --meta repo=yourorg/firmware --meta branch=main --meta
sha=a1b2c3d --wait
```

Dry-run the pipeline — proves the Coordinator↔Client plumbing works without real hardware, a real emulator image, or a reachable Artifactory:

```
thub run --type sw --image https://does-not-exist.invalid/app.bin \
  --tests https://does-not-exist.invalid/tests.tar.gz --suite
```

```
smoke \
  --dry-run --wait
```

Run on a specific resource group only:

```
thub run --type sw --image "$IMAGE_URL" --tests "$TESTS_URL" \
  --group 548ae4ae-ac5b-401f-acaa-24bbe790e62d --wait
```

Label a job with its owner — purely informational, shows up on the dashboard, in **thub jobs**, and on the Client's own console:

```
thub run --type hw --board nucleo-f401re \
  --image "$IMAGE_URL" --tests "$TESTS_URL" --user "Your Name" --
wait
```

GitHub Actions

```
test-sw:
  needs: build
  runs-on: ubuntu-latest
  env:
    THUB_URL: https://thub.example.com
    THUB_TOKEN: ${ secrets.THUB_AGENT_TOKEN }
  steps:
    - run: |
        npx -y @andrian.yablonsky/thub-agent run --type sw \
          --image "${ needs.build.outputs.image_url }" \
          --tests "${ needs.build.outputs.tests_url }" --suite
full --wait
```

If the GitHub job is canceled, the runner sends **SIGINT** to the Agent. In **--wait** mode (CI), that's treated as a cancel request (**POST /jobs/:id/cancel**) before exiting, so abandoned CI jobs don't hold hardware. In interactive mode, Ctrl-C only detaches.

Development

```
npm install
npm run lint
```

License

Proprietary — see the header comment in each source file.